```
(requires: x>= 0)
        let rec even x :=
        x=0 || odd (x-1)
        return 'even'
(requires: x>= 0)
        let odd x :=
        x<>0 && even (x-1)
        return 'odd'
```

The syntax for function types:

$$x \rightarrow y$$
$$x1 \rightarrow x2 \rightarrow y$$
$$x1 \rightarrow \ldots \rightarrow xn \rightarrow y$$

The *'x'* and *'y'* are variables indicating types. ***The type x → y is the type of a function that gets an input of type 'x' and returns an output of type 'y'.*** Whereas ***x1 → x2 → y*** is a type of a function that takes two inputs, the first input is of type ***'x1'*** and the second input of type ***'x2'***, and returns an output of type ***'y'.*** Likewise ***x1 → … → xn → y*** has type ***'x'*** as input of n arguments and ***'y'*** type as output.

Note

All functions are static definitions. There is no dynamic function definitions.

1.3 Interface Vs Implementation

An interface is a set of action that an object can do. For example when you press a light switch, the light goes on, you may not have cared how it splashed the light. In Object Oriented Programming language, an Interface is a description of all functions that a class must have in order to be a new interface. In our example, anything that ***"ACTS LIKE"*** a light, should have function definitions like turn_on () and a turn_off (). The purpose of interface is to allow the computer to enforce the properties of the class of ***TYPE T*** *(whatever the interface is)* must have functions called *X, Y, Z,* etc.

A class declaration combines the external interface *(its local state)* with an implementation of that interface *(the code that carries out the behaviour)*. An object is an instance created from the class.

The interface defines an object's visibility to the outside world.

The difference between interface and implementation is

Interface	Implementation
Interface just defines what an object can do, but won't actually do it	Implementation carries out the instructions defined in the interface

In object oriented programs classes are the interface and how the object is processed and executed is the implementation.

1.3.1 Characteristics of interface

- The class template specifies the interfaces to enable an object to be created and operated properly.
- An object's attributes and behaviour is controlled by sending functions to the object.

For example, let's take the example of increasing a car's speed.

XII Std - CS EM Chapter-1.indd 3 24-01-2020 12:38:55